

The Anthropic Claude Ecosystem

~/claude • Claude Code • Claude Desktop • Agent Skills

Vibe Coding Edition 🎸

What We'll Cover

Our Journey Today



~/claude

The config directory powering everything



Claude Code

Agentic coding across all surfaces



Claude Desktop

Your local MCP host & assistant



Agent Skills

Modular capabilities with progressive loading



Best Practices

Writing Skills that actually work

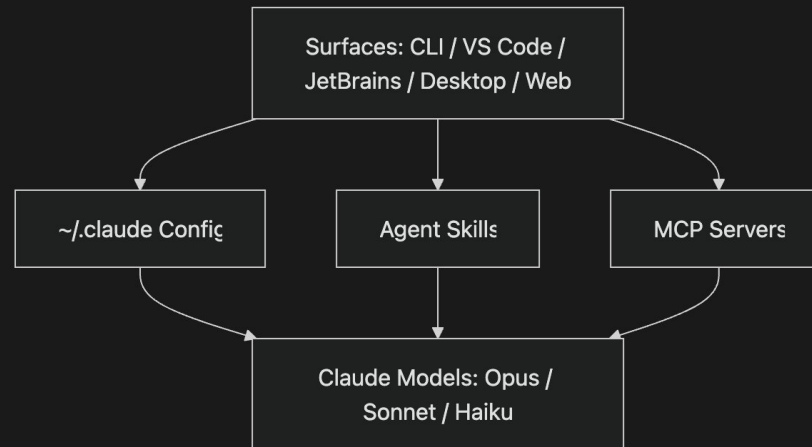


Vibe Coding

Putting it all together in real workflows

The Claude Products Landscape

One Ecosystem, Many Surfaces



All surfaces are powered by the same ~/.claude config and Claude models

Why This Matters for Vibe Coding

Staying in Flow

✗ Without the Ecosystem

Copy-pasting context every conversation

Re-explaining your codebase style manually

Switching tools constantly breaks flow

No memory of your preferences

Starting from zero each session

✓ With the Ecosystem

CLAUDE.md carries your context
automatically

Skills provide domain expertise on-demand

One tool, all surfaces (terminal → IDE → web)

Persistent preferences via ~/.claude

AI that knows your codebase deeply

Prerequisites

What You Need

- Claude Account - claude.ai (Pro/Max/Team/Enterprise for best experience)
- Claude Code installed - ``npm install -g claude-code`` or via Homebrew
- Claude Desktop - Download from claude.ai/desktop (optional but recommended)
- An API key - From console.anthropic.com for SDK work
- Node.js 18+ - Required for Claude Code CLI
- Curiosity - We'll be exploring the bleeding edge of AI-assisted coding 🚀

~/.claude

Your Personal AI Configuration Directory

The heart of your Claude setup 🏠

~/.claude Directory Structure

Your Personal AI Home

```
~/.claude/
├── CLAUDE.md           # Your global instructions for Claude
├── settings.json       # Global settings & preferences
├── commands/          # Custom slash commands
│   ├── review.md      # /review command
│   ├── test.md        # /test command
│   └── deploy.md       # /deploy command
├── skills/            # Personal Agent Skills
│   ├── my-workflow/
│   │   └── SKILL.md
│   └── company-style/
│       └── SKILL.md
├── agents/            # Custom sub-agent definitions
│   └── code-reviewer.md
```

CLAUDE.md - Your Global Instructions

Write Once, Apply Everywhere

What to Include

- Your coding style preferences
- Languages and frameworks you use
- Communication style preferences
- Project-agnostic conventions
- Things Claude should always/never do
- Your name and context

Example

My Claude Instructions

```
I'm Burin, a Clojure/ClojureScript developer.  
Prefer functional programming patterns.  
Use kebab-case for everything.  
Always write tests with clojure.test.  
Prefer concise, idiomatic Clojure.  
When in doubt, ask before refactoring.
```

Project-Level CLAUDE.md

Project-Specific Context

Global (~/.claude/CLAUDE.md)

Your personal preferences

Communication style

Language/framework defaults

Things true for all your projects

Loaded in every session automatically

Project (./CLAUDE.md)

Architecture decisions for this project

Team conventions and standards

Important file locations

Testing requirements

Deployment instructions

~/.claude/settings.json

Global Configuration

```
{
  "model": "claude-opus-4-5-20251101",
  "theme": "dark",
  "autoUpdates": true,
  "permissions": {
    "allowedTools": ["Bash", "Edit", "Write", "Glob"],
    "blockedPaths": ["/etc", "/usr/bin"]
  },
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/me/projects"]
    }
  },
  "hooks": {
    "PreToolUse": [{"matcher": "Bash", "hooks": [{"type": "command", "command": "echo 'Running bash c"}]}
  }
}
```

Custom Slash Commands

Reusable Prompt Shortcuts

Creating a Command

```
# ~/.claude/commands/review.md
```

Please do a thorough code review of the changes I've made. Focus on:

1. Logic correctness and edge cases
2. Performance implications
3. Test coverage
4. Code style and conventions
5. Security considerations

Be direct and specific. Use inline references to the actual code.

Using Commands

- Type ``/review`` in Claude Code to run it
- Commands appear in autocomplete
- Can reference `$ARGUMENTS` placeholder
- Project commands: ``.claude/commands/``
- Use Bash to generate dynamic content: ``${git diff HEAD}``

Personal Skills in ~/.claude/skills/

Global Domain Expertise

Directory Structure

```
~/.claude/skills/  
├── clojure-style/  
│   ├── SKILL.md          # Main instruction  
│   └── examples.md       # Code examples  
├── transit-viz/  
│   ├── SKILL.md  
│   └── gtfs-reference.md  
└── raylib-clj/  
    ├── SKILL.md  
    └── api-reference.md
```

Scope: Personal vs Project

- ~/.claude/skills/ → Available in ALL your projects
- .claude/skills/ → Only for this project
- Can also use Claude Code Plugins for team sharing
- Skills load on-demand, zero startup cost

Hooks System

Automation at Every Step

Hook Points

- PreToolUse - Before any tool runs
- PostToolUse - After tool completes
- Notification - When Claude sends a message
- Stop - When a session ends
- SubagentStop - When a sub-agent ends

Example Use Cases

- Auto-run linter before file writes
- Log all bash commands for audit
- Notify Slack when long task completes
- Block edits to protected files
- Auto-format after every edit

Claude Code

Agentic Coding Across Every Surface

Terminal • VS Code • JetBrains • Desktop • Browser 🚀

What is Claude Code?

More than Code Completion

- 🤖 Agentic coding tool - reads your codebase, edits files, runs commands
- 🔗 Multi-file understanding - not just autocomplete, genuine comprehension
- 🛠 Multi-surface - terminal CLI, VS Code, JetBrains, Desktop app, browser
- 📁 Git native - stages changes, writes commit messages, creates PRs
- 🔧 MCP support - connect to any external tool via Model Context Protocol
- 🎯 Fully agentic - give it a task, it plans and executes autonomously

Installation & First Run

Get Up and Running

Installation

```
# Via npm (recommended - auto-updates)
npm install -g claude-code

# Via Homebrew (macOS)
brew install claude-code

# Via WinGet (Windows)
winget install Anthropic.ClaudeCode
```

First Run

```
# Start in any project directory
cd ~/dev/my-project
claude

# You'll be prompted to log in on first use
# Then you're ready to go!

# Quick examples:
claude 'what does this codebase do?'
claude 'fix the failing tests'
claude 'add dark mode support'
```

Claude Code Surfaces

Choose Your Environment

Terminal CLI

Full-featured CLI. Edit files, run commands, manage entire projects from command line

VS Code / Cursor

Inline diffs, @-mentions, plan review, conversation history in your editor

JetBrains

Plugin for IntelliJ IDEA, PyCharm, WebStorm with diff viewing

Desktop App

Visual diff review, multiple sessions side-by-side, cloud sessions

Web Browser

No local setup. Long-running tasks, repos you don't have locally, parallel tasks

CI/CD

GitHub Actions, GitLab CI - automate code review and issue triage

Terminal CLI

The Full-Power Surface

Key Commands

```
# Interactive mode (default)
claude

# One-shot with prompt
claude 'explain this codebase'

# Pipe input
cat error.log | claude 'what went wrong?'

# Continue last conversation
claude --continue

# Resume specific session
claude --resume <session-id>

# List custom agents
claude agents
```

In-Session Commands

- `/help` - Show available commands
- `/clear` - Clear conversation history
- `/model` - Switch Claude model
- `/mcp` - Manage MCP connections
- `/memory` - View/edit memories
- Custom commands from
`~/.claude/commands/`

VS Code Extension

Claude Inside Your Editor

Key Features

- Inline diffs with accept/reject
- @-mentions for files and symbols
- Plan review before execution
- Conversation history panel
- Integrated terminal sessions
- Works with Cursor too!

Setup

```
# Install via Command Palette  
Cmd+Shift+X → search 'Claude Code'  
  
# Or for Cursor:  
Search 'Claude Code' in Extensions  
  
# After install:  
Cmd+Shift+P → 'Claude Code'  
→ 'Open in New Tab'
```

Common Claude Code Workflows

What It's Great At



Tedious Tasks

- Writing tests for untested code
- Fixing lint errors across a project
- Resolving merge conflicts
- Updating dependencies
- Writing release notes
- Reformatting/refactoring code style

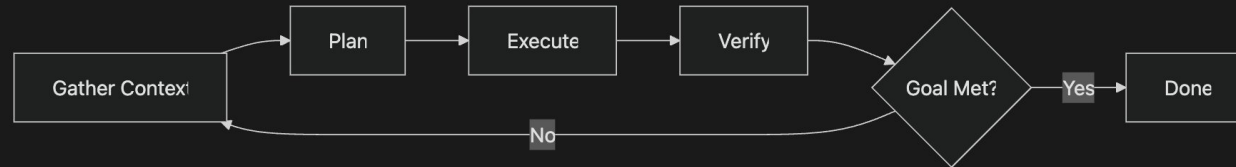


High-Level Work

- Building features end-to-end
- Debugging from error messages
- Git workflows (branches, PRs, commits)
- Code review and analysis
- Documentation generation
- Architecture exploration

The Agentic Loop

How Claude Code Reasons and Acts



Claude Code autonomously loops until the task is complete

Git Integration

Goodbye to Git Busywork

What Claude Code Does

- Stages the right changes automatically
- Writes descriptive commit messages
- Creates feature branches with good names
- Opens pull requests with descriptions
- Resolves merge conflicts intelligently
- Understands git history for context

Example Session

```
> add user authentication with JWT

[Claude reads codebase]
[Creates auth middleware]
[Updates routes]
[Adds tests]
[Stages relevant files]
[Commits: 'feat(auth): implement JWT...'
  'Add login endpoint and token validation']
[Creates PR with full description]
```

MCP in Claude Code

Extending Claude with External Tools

Configure in settings.json

```
// ~/.claude/settings.json
{
  "mcpServers": {
    "github": {
      "command": "uvx",
      "args": ["mcp-server-github"],
      "env": {
        "GITHUB_TOKEN": "ghp_..."
      }
    },
    "postgres": {
      "command": "npx",
      "args": ["mcp-server-postgres",
        "postgresql://localhost/mydb"]
    }
  }
}
```

Popular MCP Servers

@mcp/filesystem - Local file access

@mcp/github - GitHub integration

@mcp/postgres - Database queries

@mcp/slack - Slack messaging

@mcp/puppeteer - Web automation

clojure-mcp - Clojure REPL!

CI/CD Integration

Claude in Your Pipeline

```
# .github/workflows/claude-review.yml
name: Claude Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Claude Code Review
        uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: |
            Review this PR for:
            - Logic errors and bugs
            - Security vulnerabilities
            - Test coverage gaps
            - Code style consistency
```

CLAUDE_CODE_SIMPLE Mode

Minimal & Fast

What It Disables

- MCP tools and servers
- File attachments
- Hook execution
- CLAUDE.md file loading
- Session memory
- Custom agents and Skills
- Token counting for CLAUDE.md

When to Use

```
# Enable SIMPLE mode
export CLAUDE_CODE_SIMPLE=1
claude

# Good for:
# - Quick one-off questions
# - Debugging Claude Code itself
# - Speed-critical pipelines
# - Minimal token usage
```

Claude Code Analytics API

Measure Your Team's AI Usage

- 📊 Daily aggregated metrics per developer
- 💻 Sessions tracked - how often devs use Claude Code
- 📝 Lines of code added/removed via Claude
- 🔗 Git metrics - commits and PRs created through Claude
- ⚙️ Tool acceptance rates - which suggestions developers accept
- 💰 Token and cost data - broken down by Claude model
- 📈 Build custom dashboards via `/v1/organizations/usage_report/claude_code``

Cloud vs Local Sessions

Flexibility of Execution

Local Sessions

Runs on your machine

Full filesystem access

Your local tools & environment

No upload required

Works offline (mostly)

Best for active development

Cloud Sessions

Runs on Anthropic infrastructure

No local setup needed

Long-running background tasks

Work on repos not locally available

Multiple parallel sessions

Start from claude.ai/code

Claude Desktop

Your Local AI Hub & MCP Host

The best home for your personal AI assistant 🖥️

Claude Desktop

More Than a Chat Interface



MCP Host

Connect to MCP servers for filesystem, databases, APIs, and custom tools



Claude Code Tab

Visual diff review, multiple coding sessions side-by-side



Artifacts

Create and preview code, documents, images, and interactive content



Agent Skills

Pre-built Skills for PowerPoint, Excel, Word, PDF + your custom Skills



File Access

Upload files directly or use MCP filesystem server for project access



Projects

Organize conversations by project with shared context

Setting Up Claude Desktop

Installation & Configuration

Install

- Download from claude.ai/desktop
- Available for macOS and Windows
- Launch → Sign in with Claude account
- Subscription required (Pro/Max/Team)
- Click Code tab for Claude Code features

Config File Location

```
# macOS
~/Library/Application Support/Claude/
└─ claude_desktop_config.json

# Windows
%APPDATA%\Claude\
└─ claude_desktop_config.json
```

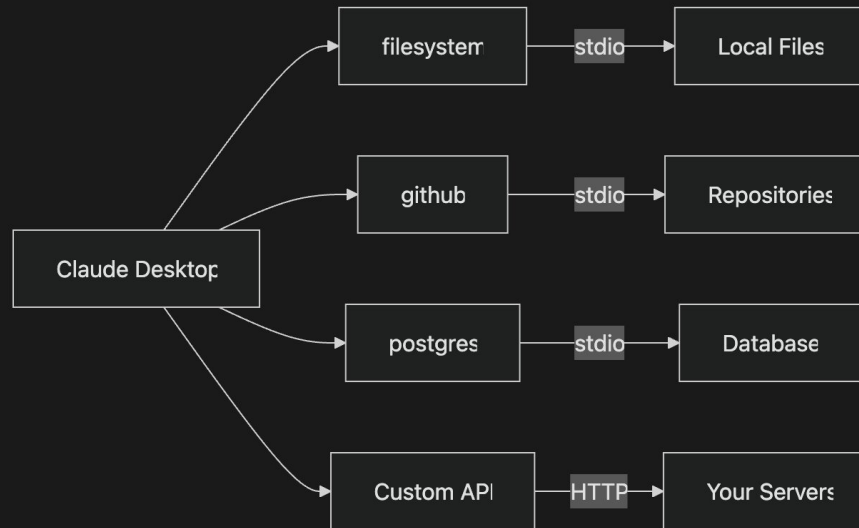
Claude Desktop as MCP Host

Extending Claude with Local Capabilities

```
// ~/Library/Application Support/Claude/claude_desktop_config.json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
        "/Users/me/dev", "/Users/me/Documents"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..." }
    },
    "clojure-mcp": {
      "command": "clj",
      "args": ["-M:mcp"]
    }
  }
}
```

Claude Desktop + MCP Architecture

Claude as the Connected Hub



Each MCP client connects to one server via stdio or HTTP transport

MCP Transport Types

stdio vs Streamable HTTP

stdio (Standard IO)

Claude launches the server process

Communication via stdin/stdout

Best for local tools

No network required

Process lifecycle managed by Claude

Most MCP servers use this

Streamable HTTP

Server runs independently

Communication via HTTP + SSE

Best for remote services

Can be shared across clients

Supports authentication

The future for cloud MCP

Agent Skills in Claude Desktop

Built-in Superpowers

Pre-built Skills (Active by Default)

PowerPoint - Create & edit presentations

Excel - Spreadsheets and data analysis

Word - Document creation and editing

PDF - Generate formatted PDF reports

Available to all users on claude.ai

No setup required - just ask!

Custom Skills (Upload Your Own)

Upload via Settings > Features

Upload as a zip file

Available on Pro/Max/Team/Enterprise

Requires code execution enabled

Individual user only (not org-wide)

Package domain expertise & workflows

Projects

Organized Context for Different Work

What Projects Provide

- Dedicated conversation workspace
- Project-specific context and instructions
- Shared files within the project
- Team sharing (on Team/Enterprise)
- Separate from personal conversations
- Custom project-level system prompt

Use Case Examples

- Client A's codebase - with their conventions
- Personal side projects - your style
- Team research project - shared documents
- Learning a new framework - reference docs
- Writing project - style guide and drafts

Agent Skills

Modular Capabilities with Progressive Loading

Anthropic's Official Documentation Deep Dive 📖

Agent Skills

From the Official Anthropic Documentation

Agent Skills are modular capabilities that extend Claude's functionality. Each Skill packages instructions, metadata, and optional resources (scripts, templates) that Claude uses automatically when relevant.

-  Reusable - filesystem-based resources, create once use many times
-  Domain-specific - workflows, context, best practices for a specialty
-  On-demand - load only when relevant, zero startup cost
-  Composable - combine Skills for complex workflows
-  Source - platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

Skills vs Prompts

When to Use Each

Prompts

Conversation-level instructions

One-off tasks

Needs to be repeated each session

Simple guidance, no code

In-context at all times

Good for: single-use customization

Skills

Filesystem-based packages

Load on-demand when relevant

Never need to repeat instructions

Can include code and resources

Progressive loading - minimal cost

Good for: reusable domain expertise

How Skills Work

Progressive Disclosure Architecture

1 Level 1: Metadata

YAML frontmatter. ~100 tokens
per Skill. ALWAYS loaded at
startup

2 Level 2: Instructions

SKILL.md body. Under 5k
tokens. Loaded WHEN triggered
by relevance

3 Level 3: Resources

Bundled files & scripts.
Effectively unlimited. Loaded AS
NEEDED

Level 1: Metadata

Always Loaded - Zero Token Cost for Many Skills

What It Is

```
# SKILL.md frontmatter
---
name: pdf-processing
description: Extract text and tables from
  PDF files, fill forms, merge documents.
  Use when working with PDF files or when
  the user mentions PDFs, forms, or
  document extraction.
---
```

How It Works

- Loaded into system prompt at startup
- Only ~100 tokens per Skill
- Can have 100+ Skills with minimal cost
- Claude uses description to decide when to trigger
- This is the Skill's 'calling card'

Level 2: Instructions

Loaded When Triggered - The Skill's Main Content

SKILL.md Body

```
---
name: pdf-processing
description: Extract text from PDFs...
---

# PDF Processing

## Quick start

Use pdfplumber for text extraction:

```python
import pdfplumber
with pdfplumber.open('file.pdf') as pdf:
 text = pdf.pages[0].extract_text()
```

For form filling, see [FORMS.md](FORMS.md)
For API reference, see [REF.md](REF.md)
```

Key Points

- Claude reads SKILL.md via bash when triggered
- Under 5k tokens recommended
- Only loaded when relevant to the request
- Points to additional files via links
- The main 'onboarding guide' for the task

Level 3: Resources & Code

Loaded As Needed - No Practical Limit

Directory Structure

```
pdf-skill/
├── SKILL.md           # Loaded when trigge
├── FORMS.md          # Loaded if form wor
├── REFERENCE.md      # Loaded for API det
├── examples.md       # Loaded for example
└── scripts/
    ├── analyze_form.py # EXECUTED not loa
    ├── fill_form.py   # Output enters co
    └── validate.py    # Code itself does
```

Three Resource Types

Instructions - Markdown files with guidance
(FORMS.md)

Code - Executable scripts (fill_form.py) - only
OUTPUT in context

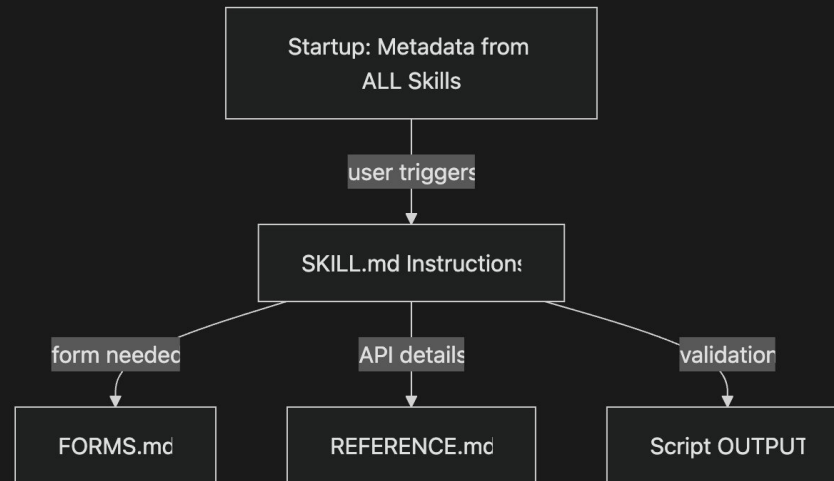
Resources - Schemas, templates, reference
data

No penalty for large bundled content

Only consumed when explicitly accessed

Progressive Disclosure in Action

Only What's Needed, When It's Needed



Scripts are executed via bash - only their OUTPUT enters the context window

Where Skills Work

Surface Availability

Claude.ai

Pre-built Skills active by default
Custom Skills via Settings > Features
Upload as zip files
Pro/Max/Team/Enterprise plans
Per-user only (not org-wide)

Claude Code

Custom Skills only (no pre-built)
Filesystem-based (no API upload)
~/.claude/skills/ for global
.claude/skills/ for project
Full network access

Claude API

Both pre-built and custom Skills
Upload via /v1/skills endpoints
Workspace-wide (all members)
No network access in sandbox
Specify skill_id in container param

Agent SDK

Custom Skills via .claude/skills/
Enable 'Skill' in allowed_tools
Auto-discovered on startup
Programmatic Python/TypeScript
Same ecosystem as Claude Code

SKILL.md Structure

The Required Format

Template

```
---
name: your-skill-name
description: Brief description of what this
  Skill does and when to use it. Include
  both what it does AND when to use it.
---

# Your Skill Name

## Instructions
[Clear, step-by-step guidance for Claude]

## Examples
[Concrete examples of using this Skill]
```

Field Constraints

- name: max 64 chars, lowercase letters/numbers/hyphens only
- name: cannot contain XML tags
- name: cannot use reserved words: 'anthropic', 'claude'
- description: max 1024 chars, cannot be empty
- description: cannot contain XML tags
- description: must include what AND when

Pre-built Agent Skills

Available to All Users



PowerPoint (pptx)

Create presentations, edit slides, analyze presentation content. Just say 'create a presentation about...'



Excel (xlsx)

Create spreadsheets, analyze data, generate reports with charts. Works on tabular data and .xlsx files



Word (docx)

Create documents, edit content, format text. Ask for reports, letters, templates



PDF (pdf)

Generate formatted PDF documents and reports. For reading PDFs, upload them directly

Security Considerations

⚠ Important - From Official Docs

We strongly recommend using Skills only from trusted sources: those you created yourself or obtained from Anthropic. Skills provide Claude with new capabilities through instructions and code, and while this makes them powerful, it also means a malicious Skill can direct Claude to invoke tools or execute code in ways that don't match the Skill's stated purpose.

- 🔍 Audit thoroughly - Review ALL bundled files including scripts
- 🌐 External sources are risky - Fetched content may contain injections
- 🛡 Tool misuse - Malicious Skills can invoke bash, file ops in harmful ways
- 📁 Data exposure - Poorly written Skills can leak sensitive data
- ⚙ Treat like software - Same trust model as npm install

Current Limitations

Plan Around These

Cross-Surface Availability

Skills DON'T sync across surfaces

Claude.ai upload $\neq$ API upload

Must upload separately for each surface

Claude Code uses filesystem (no upload)

Plan for multi-surface distribution

Runtime Constraints

Claude.ai - Varies by user/admin settings

Claude API - No network access at all

Claude API - No runtime package install

Claude Code - Full network access

Test on each surface you plan to use

Creating Your First Custom Skill

A Practical Example

```
---
name: clojure-conventions
description: Apply idiomatic Clojure coding conventions and
  style. Use when writing or reviewing Clojure code, or when
  the user asks about Clojure style, idioms, or conventions.
---

# Clojure Conventions

## Core principles

- Prefer functional over imperative patterns
- Use kebab-case for all identifiers
- Keep functions small and focused (under 20 lines)
- Use destructuring liberally
- Prefer `->` and `->>` threading macros

## Namespace organization

Require in alphabetical order. Group: stdlib, then third-party, then internal.

## Testing

Use clojure.test. Test public functions, not implementation details.
See [testing.md](testing.md) for detailed patterns.
```

Skill Best Practices

Writing Skills That Actually Work

From the Official Anthropic Best Practices Guide 📋

Core Principle: Concise is Key

The Context Window is a Public Good

❌ Too Verbose (150 tokens)

- Explains what PDFs are
- Why pdfplumber is recommended
- How to install it with pip
- General background about libraries
- All information Claude already knows

✅ Concise (50 tokens)

- Jumps straight to the code
- Assumes Claude knows what PDFs are
- No installation instructions
- No explanations of obvious things
- Just the specific guidance needed

*Default assumption: Claude is already very smart. Only add context Claude doesn't already have.
Challenge each piece of information: Does Claude really need this explanation?*

Degrees of Freedom

Match Specificity to Risk



High Freedom

Open field - many paths to success. Use text instructions. Code reviews, analysis, writing tasks



Medium Freedom

Preferred path, some variation OK. Use pseudocode/templates with parameters. Report generation



Low Freedom

Narrow bridge - one safe path. Use exact scripts, no parameters. DB migrations, critical ops

Think of Claude as a robot exploring a path. Narrow bridge with cliffs: only one safe way forward - provide exact instructions. Open field with no hazards: many paths lead to success - give general direction and trust Claude.

Writing Effective Descriptions

The Key to Skill Discovery

✗ Bad Descriptions

'Helps with documents' - too vague

'Processes data' - too generic

'I can help you process Excel files' - wrong person

'You can use this for PDFs' - wrong person

No trigger conditions mentioned

No specific use cases listed

✓ Good Descriptions

Third person (what it DOES, not what 'I' do)

Specific about what the Skill provides

Includes when to use it (triggers)

Lists specific file types or keywords

Example: 'Analyzes Excel spreadsheets, creates pivot tables. Use when analyzing Excel files, spreadsheets, tabular data, or .xlsx files.'

Naming Conventions

Consistent, Clear, Memorable

✓ Good Names (Gerund Form)

processing-pdfs
analyzing-spreadsheets
managing-databases
testing-code
writing-documentation
reviewing-clojure

✗ Avoid

'helper' - too vague
'utils' - too generic
'tools' - meaningless
'anthropic-helper' - reserved word
'claude-tools' - reserved word
Inconsistent patterns in same collection

Progressive Disclosure Patterns

Three Proven Organizational Patterns



Pattern 1: Guide + References

SKILL.md has overview.
Additional files for advanced topics. Links in SKILL.md point to details.



Pattern 2: Domain Organization

Multiple reference files by domain. Finance.md, Sales.md, Product.md. Each loaded only when relevant.



Pattern 3: Conditional Details

Show basic content. Link to advanced sections only when needed. 'For tracked changes, see REDLINING.md'

Key rule: Keep references ONE level deep from SKILL.md. All reference files should link directly from SKILL.md to ensure Claude reads complete files when needed. Avoid chaining: SKILL.md → advanced.md → details.md

Workflows & Feedback Loops

For Complex Multi-Step Tasks

Workflow Pattern

PDF form filling workflow

Copy this checklist:

- [] Step 1: Analyze form (*analyze_form.py*)
- [] Step 2: Create field mapping (*fields.*
- [] Step 3: Validate mapping (*validate.py*
- [] Step 4: Fill form (*fill_form.py*)
- [] Step 5: Verify output (*verify.py*)

Checklist prevents skipping steps!

Feedback Loop Pattern

Document editing process

1. Make your edits to *document.xml*
2. Validate immediately: *python validate.py*
3. If validation fails:
 - Review the error message carefully
 - Fix the issues
 - Run validation AGAIN
4. Only proceed when validation passes
5. Rebuild: *python pack.py doc/ output.docx*

Evaluation-Driven Development

Test Before You Document

The Process

1. Run Claude WITHOUT a Skill first
2. Document specific failures & gaps
3. Build 3+ evaluations for the gaps
4. Establish a baseline measurement
5. Write MINIMAL instructions to pass evals
6. Iterate: evaluate → compare → refine

Evaluation Structure

```
{
  "skills": ["pdf-processing"],
  "query": "Extract all text from this PDF",
  "files": ["test-files/document.pdf"],
  "expected_behavior": [
    "Successfully reads the PDF file",
    "Extracts text from all pages",
    "Saves to output.txt"
  ]
}
```

The Claude A / Claude B Method

Use Claude to Build Claude Skills

Claude A (Expert) creates/refines the Skill → Claude B (Agent) uses the Skill in real work → You observe Claude B's behavior → bring insights back to Claude A

- Step 1: Complete a task WITH Claude A using normal prompting
- Step 2: Ask Claude A: 'Create a Skill that captures what we just did'
- Step 3: Review for conciseness with Claude A
- Step 4: Test with Claude B (fresh instance with Skill loaded) on real tasks
- Step 5: If Claude B struggles, tell Claude A what happened and iterate
- Why it works: Claude understands Skill format natively - just ask it!

Anti-patterns to Avoid

Common Mistakes from the Official Guide

✗ Structural Issues

Windows paths: ``scripts\helper.py`` ← breaks on Unix

Deeply nested references: `A→B→C` (use 1 level)

No table of contents for files > 100 lines

Time-sensitive info: 'Before August 2025, use...'

Inconsistent terminology: 'field'/'box'/'element'

✗ Content Issues

Too many options: 'You can use pypdf, or pdfplumber, or...'

Explaining obvious things Claude already knows

Magic constants without justification:
``TIMEOUT = 47``

Assuming packages are installed without checking

Scripts that punt to Claude instead of handling errors

Skills Quality Checklist

From Official Best Practices Guide

Core Quality

- ✓ Description specific with key terms
- ✓ Description includes what AND when
 - ✓ SKILL.md body under 500 lines
- ✓ Additional details in separate files
 - ✓ No time-sensitive information
- ✓ Consistent terminology throughout
- ✓ Examples are concrete, not abstract
 - ✓ References are one level deep

Testing & Code

- ✓ At least 3 evaluations created
 - ✓ Tested with target models
- ✓ Tested with real usage scenarios
- ✓ Scripts solve problems (not punt)
 - ✓ Error handling is explicit
- ✓ No Windows-style paths
- ✓ Required packages listed and available
- ✓ Feedback loops for critical operations

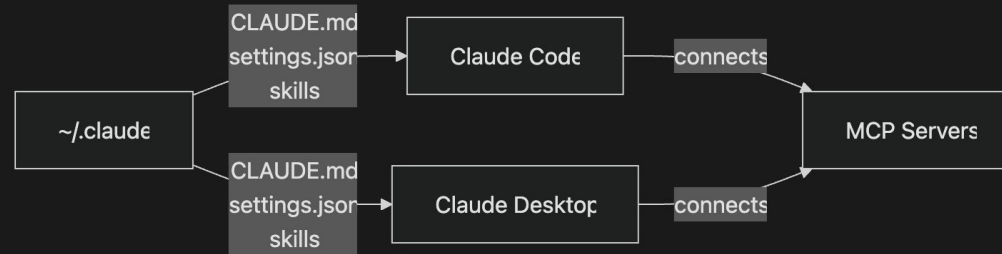
Putting It All Together

The Complete Claude Ecosystem for Vibe Coding

Real workflows that keep you in the zone 🎯

The Complete Ecosystem

Everything Connected



The ~/claude directory is your persistent AI configuration layer

A Day in the Life

Full Ecosystem Vibe Coding

- 🌅 Morning - Open Claude Desktop, chat about today's architecture decisions with full codebase context via MCP filesystem
- 💻 Feature work - Fire up `claude` in terminal, give high-level task: 'Add OAuth login', it plans + executes autonomously
- 🔍 Code review - VS Code extension shows inline diffs, accept/reject each change, Claude Code refines
- 📝 Documentation - Ask Claude Desktop to generate docs using Word Skill, it creates proper .docx output
- 🐛 Debugging - Paste error in Claude Code: 'fix this', it reads stack trace, traces through codebase, proposes fix
- 🚀 Deploy - Run `/deploy` custom slash command which runs your CI checks then creates the PR
- 📊 Reporting - Claude Desktop generates Excel report using Skills from the day's metrics

Getting Started Tips

From Zero to Productive

Week 1: Essentials

Install Claude Code: ``npm i -g claude-code``

Write your `~/claude/CLAUDE.md`

Add 1-2 MCP servers to `settings.json`

Try: ``claude 'explain this codebase'``

Create your first custom slash command

Week 2: Level Up

Add a project `CLAUDE.md` to your main project

Create your first custom Skill

Explore the VS Code extension

Set up GitHub MCP for PR creation

Try giving Claude a complex multi-file task

Week 3: Master

Build a Skills library for your domain

Configure hooks for automated checks

Set up Claude Code in your CI/CD pipeline

Create team-shared Skill packages

Explore the Agent SDK for custom automations

Long Term

Iterate Skills based on real usage

Share Skills with your team

Build custom MCP servers for your stack

Automate your entire workflow

Contribute to the MCP ecosystem!

Resources & References

Everything You Need to Continue

Official Documentation

- Claude Code -
code.claude.com/docs/en/overview
- Agent Skills Overview -
platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
- Skills Best Practices -
platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
- Agent SDK -
platform.claude.com/docs/en/agent-sdk/overview
- MCP Protocol - modelcontextprotocol.io

Community & Tools

- MCP Servers -
github.com/modelcontextprotocol/servers
- Skills Cookbook -
platform.claude.com/cookbook/skills-notebooks-01
- Discord - anthropic.com/discord
- GitHub - github.com/anthropics
- Claude Status - status.claude.com

Questions?

Let's Discuss, Explore, and Vibe Code Together



Burin Choomnuan

Principal Engineer at News Corp, Sydney. 20+ years full stack. Clojure, Python, and Lisp enthusiast

LinkedIn



linkedin.com/in/burinc

GitHub



github.com/burinc